

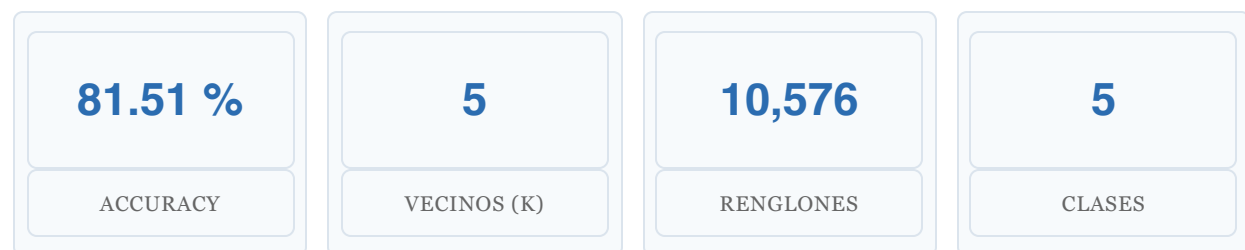
Clasificación del tipo de garantía con K-Vecinos Más Cercanos

Implementación de una técnica de aprendizaje máquina sin framework

Momento de Retroalimentación · Módulo 2 · TC3009 | Subcompetencia SMA0401A

1. Resumen

Se implementó desde cero el algoritmo **K-Vecinos Más Cercanos (KNN)** para clasificar el tipo de garantía de un crédito automotriz a partir de los registros mensuales de intereses de una cartera real. La implementación completa vive en un solo archivo `main.py` y no utiliza ninguna biblioteca de aprendizaje máquina ni de estadística: la media, la desviación estándar, la distancia euclidiana, la votación de vecinos, la matriz de confusión y las métricas de precisión, recall y F1 están programadas manualmente. Matplotlib se emplea únicamente para graficar los resultados ya calculados.



El modelo alcanza **81.51 % de accuracy** sobre el conjunto de prueba, muy por encima del 43 % que obtendría una regla trivial que siempre respondiera la clase mayoritaria. El desempeño es excelente en las clases separables por monto y antigüedad (F1 de 0.98 y 0.95) y débil en la clase minoritaria DEMO, por las razones que se analizan en la sección 6.

2. El algoritmo: cómo funciona KNN

KNN es un método de *aprendizaje perezoso*: no construye un modelo durante el entrenamiento. Entrenar consiste únicamente en memorizar los ejemplos vistos, y todo el trabajo ocurre al momento de predecir. Para clasificar una observación nueva:

1. Se mide la distancia de esa observación a **cada uno** de los puntos de entrenamiento.
2. Se ordenan las distancias y se toman los **k puntos más cercanos** (los vecinos).
3. Los vecinos **votan**; la clase con más votos es la predicción.

La idea de fondo es la hipótesis de continuidad: observaciones parecidas en las variables de entrada tienden a pertenecer a la misma clase. La distancia empleada es la euclidiana, implementada

directamente con la fórmula:

```
def distancia_euclidiana(a, b):  
    suma = 0  
    for i in range(len(a)):  
        suma += (a[i] - b[i]) ** 2  
    return math.sqrt(suma)
```

2.1 La normalización es indispensable

La distancia euclidiana suma las diferencias de todas las variables. En este dataset el `principal` se mide en cientos de miles de pesos mientras que la `tasa` vive entre 0 y 0.01. Sin normalizar, la primera variable domina por completo la suma y la segunda desaparece del modelo. Por eso cada variable se transforma a media 0 y desviación estándar 1 (puntuación z) antes de medir distancias:

$$z = (x - media) / desviación$$

Prevención de fuga de información. La media y la desviación se calculan **únicamente con los datos de entrenamiento** y después se aplican al conjunto de prueba. Calcularlas sobre el archivo completo sería fuga de información: el modelo estaría usando datos del examen para prepararse, y las métricas saldrían infladas.

3. Dataset

El archivo `data/interest_ledger.csv` contiene **10,576 renglones** del libro contable de intereses de una cartera de crédito automotriz. Cada renglón representa el interés devengado por un vehículo durante un mes contable.

3.1 Variable objetivo

La columna `collateral` indica el tipo de garantía del crédito, con cinco clases posibles. La distribución está fuertemente desbalanceada, hecho determinante para interpretar los resultados:

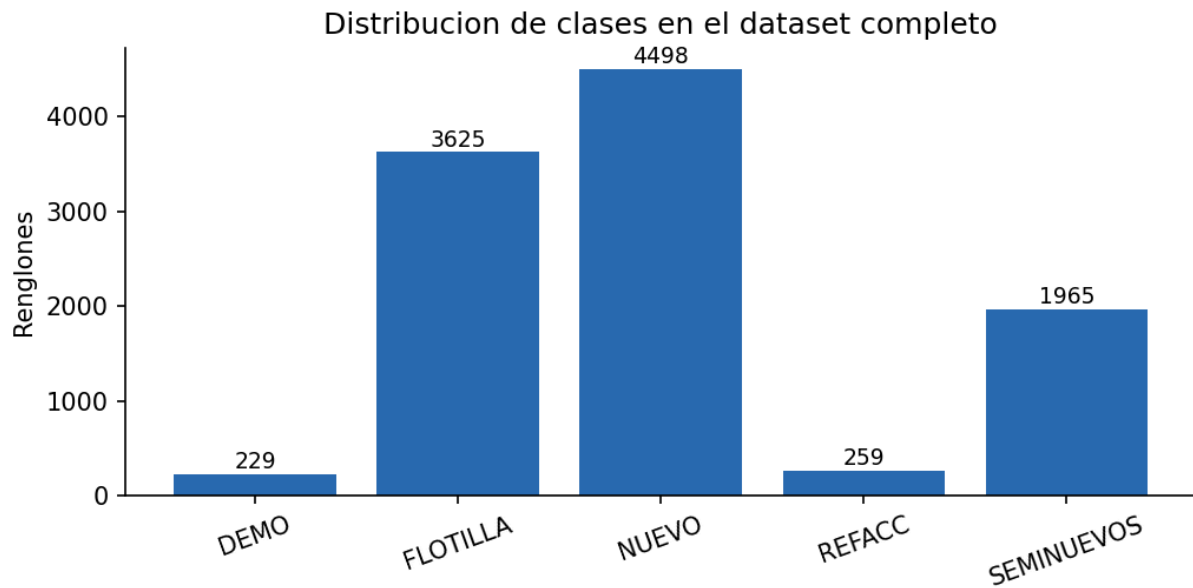


Figura 1. Distribución de clases. NUEVO tiene casi 20 veces más casos que DEMO.

3.2 Variables de entrada

De cada renglón se derivan cinco variables numéricas:

Variable	Descripción	Origen
principal	Monto del crédito vigente en el mes	Directa
interes	Interés devengado durante el mes	Directa
tasa	Interés por cada peso prestado	interes / principal
antiguedad	Años entre el modelo del vehículo y el mes contable	año contable – año modelo
mes	Mes del año, capta estacionalidad	De la fecha

La columna `collateral` del export original traía espacios no separables (\xa0) pegados a cada valor, de modo que "NUEVO\xa0" y "NUEVO" se leían como clases distintas. La carga de datos los limpia antes de usar la etiqueta.

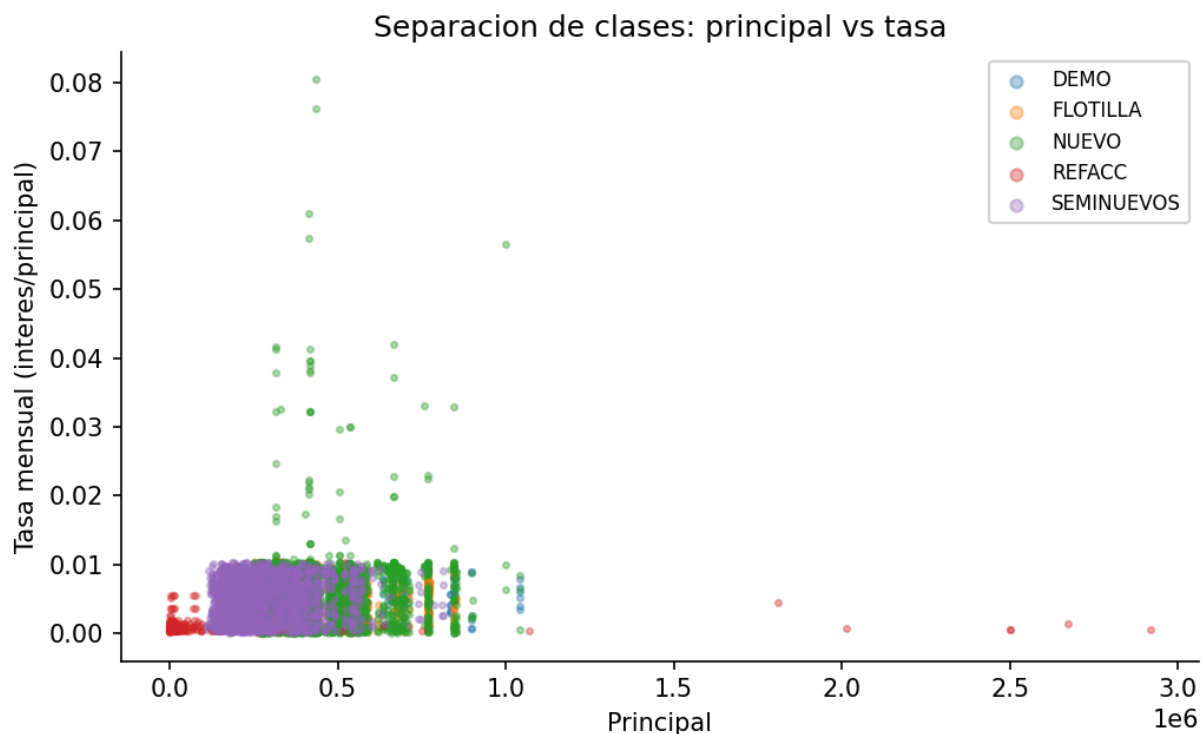


Figura 2. Principal contra tasa mensual. REFACC ocupa la zona de montos bajos y SEMINUEVOS se separa por antigüedad, mientras NUEVO y FLOTILLA se traslapan casi por completo.

3.3 Partición entrenamiento / prueba

Los datos se dividen aleatoriamente en **75 % entrenamiento y 25 % prueba**, con semilla fija (`random.seed(42)`) para que el resultado sea reproducible en cualquier ejecución.

Conjunto	Renglones	Proporción	Uso
Entrenamiento	7,932	75 %	Memorizado por el modelo; define media y desviación
Prueba	2,644	25 %	Nunca visto durante el ajuste; sirve para medir

4. Métricas utilizadas

Con clases tan desbalanceadas el accuracy por sí solo engaña. Un modelo que respondiera siempre "NUEVO" lograría cerca del 43 % sin haber aprendido absolutamente nada. Por eso se reportan además tres métricas por clase, todas calculadas a mano a partir de la matriz de confusión:

Métrica	Fórmula	Qué responde
Precisión	$TP / (TP + FP)$	De todo lo que predije como esta clase, ¿cuánto acerté?
Recall	$TP / (TP + FN)$	De todos los casos que realmente eran de esta clase, ¿cuántos encontré?
F1	$2 \cdot P \cdot R / (P + R)$	Balance entre ambas; penaliza que una sea alta a costa de la otra

La distinción importa: un modelo puede tener precisión alta y recall bajo si predice una clase pocas veces pero casi siempre acierta, lo cual es exactamente lo que ocurre con DEMO.

5. Resultados

5.1 Matriz de confusión

La matriz cruza la clase real (renglones) contra la predicción (columnas). La diagonal contiene los aciertos; todo lo que queda fuera son errores, y su posición indica exactamente qué clase se confundió con cuál.

Real \ Predicho	DEMO	FLOTILLA	NUEVO	REFACC	SEMINUEVOS
DEMO	7	18	25	0	0
FLOTILLA	6	692	151	0	5
NUEVO	11	227	933	0	8
REFACC	0	0	2	57	0
SEMINUEVOS	1	19	16	0	466

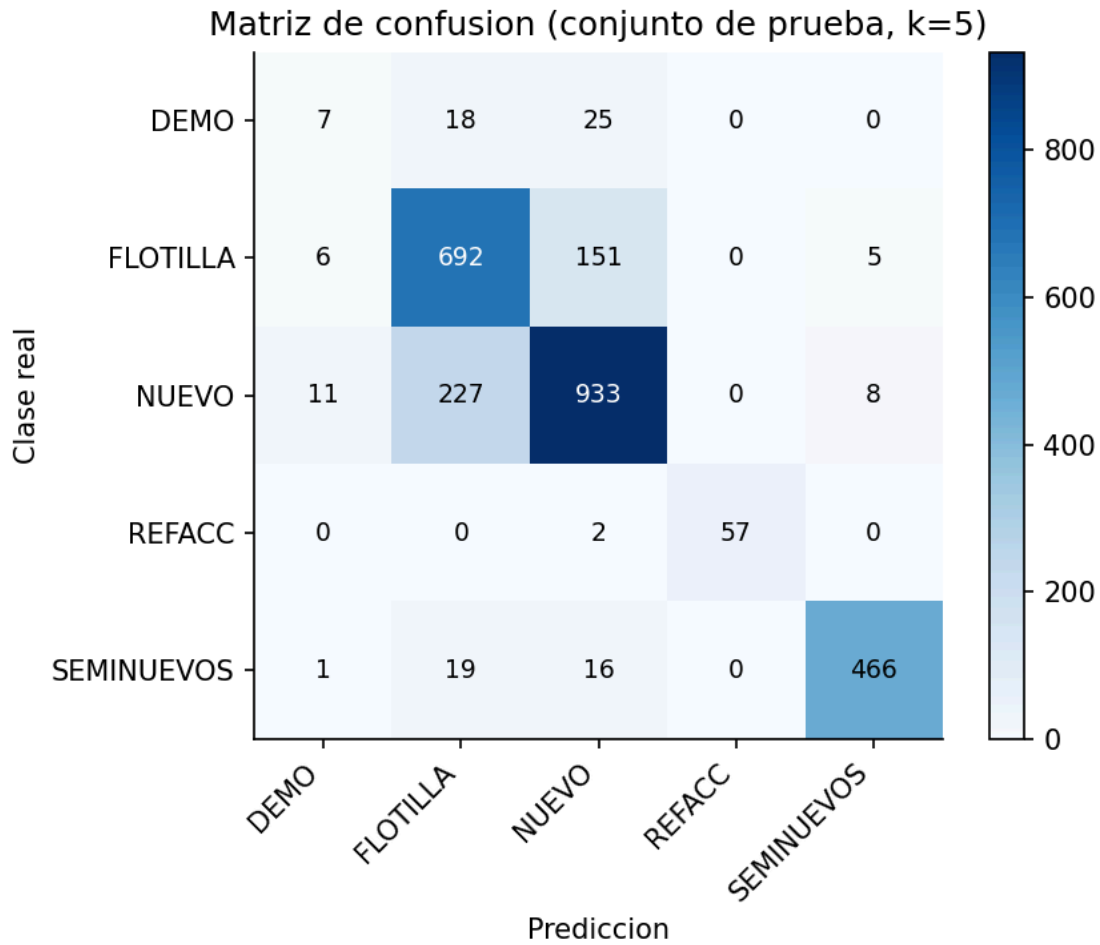


Figura 3. Matriz de confusión. El bloque intenso fuera de la diagonal es la confusión mutua entre NUEVO y FLOTILLA.

5.2 Métricas por clase

Clase	Precisión	Recall	F1	Soporte
DEMO	0.280	0.140	0.187	50
FLOTILLA	0.724	0.810	0.765	854
NUEVO	0.828	0.791	0.809	1,179
REFACC	1.000	0.966	0.983	59
SEMINUEVOS	0.973	0.928	0.950	502
Accuracy global: 81.51 % (2,155 aciertos de 2,644 casos)				

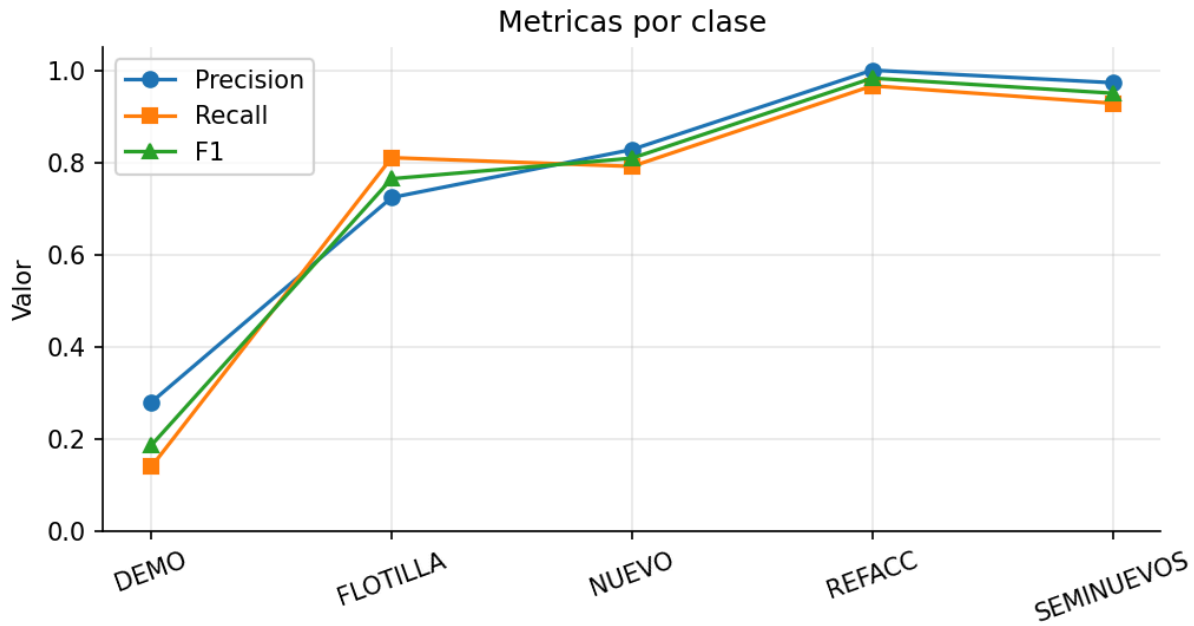


Figura 4. Métricas por clase. El contraste entre REFACC/SEMINUEVOS y DEMO es la consecuencia directa del desbalance.

5.3 Efecto del número de vecinos

Se evaluó el modelo con distintos valores de k sobre el mismo conjunto de prueba:

k	1	3	5	7	9	11	15	21
Accuracy (%)	84.23	82.75	81.51	81.01	80.22	80.48	79.84	79.01

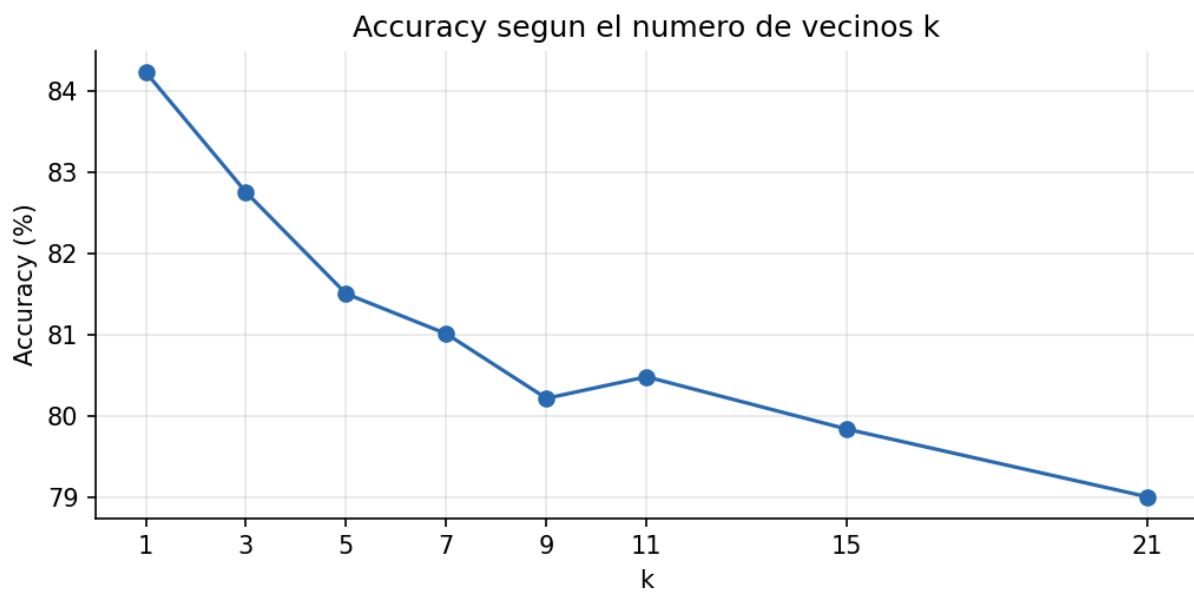


Figura 5. Accuracy contra k . La curva desciende de forma sostenida al aumentar el número de vecinos.

Por qué $k = 1$ obtiene el mejor número y aun así no se eligió. El dataset contiene 2.58 renglones por vehículo en promedio (10,576 renglones para 4,092 VIN distintos), y el 95.2 % de los vehículos conserva la misma garantía todos los meses. Con una partición aleatoria por renglón, el vecino más cercano de un caso de prueba suele ser *el mismo vehículo en otro mes contable*. Con $k = 1$ el modelo no está generalizando: está reconociendo vehículos que ya vio. Ese 84.23 % es optimista y no representa el desempeño ante un vehículo nuevo. Se eligió $k = 5$ porque promedia varios vecinos y reduce esa dependencia, a costa de un accuracy reportado más honesto.

6. Análisis

6.1 Dónde está el error

La matriz de confusión revela que el error no está repartido: se concentra en un solo lugar. Los 227 casos de NUEVO clasificados como FLOTILLA más los 151 de FLOTILLA clasificados como NUEVO suman 378 errores, el **77 % de los 489 errores totales**. El resto del modelo funciona bien.

Esta confusión es esperable y no indica una falla de la implementación. Un vehículo de flotilla *también* es un vehículo nuevo: la diferencia entre ambas categorías no está en el monto, la tasa ni la antigüedad, sino en la naturaleza del cliente (persona física contra persona moral que compra varias unidades). Como esa información no forma parte de las variables de entrada, los dos grupos ocupan prácticamente la misma región del espacio y ningún clasificador basado en distancias podría separarlos con los datos disponibles. La figura 2 lo muestra gráficamente: sus nubes de puntos se traslapan.

6.2 Las clases que sí se separan

REFACC alcanza precisión perfecta (1.000) y recall de 0.966: los créditos de refacciones tienen montos mucho menores que los de vehículos, así que quedan aislados en el espacio y ningún otro caso cae cerca. **SEMINUEVOS** logra F1 de 0.950 porque la antigüedad del modelo lo distingue con claridad de todo lo demás. Cuando la información necesaria está presente en las variables, el algoritmo la aprovecha correctamente.

6.3 El problema de la clase minoritaria

DEMO es la clase débil, con recall de apenas 0.140: de 50 casos reales en prueba, el modelo solo identificó 7. La causa es estructural y propia de KNN. Con 229 casos en todo el dataset contra 4,498 de NUEVO, los puntos de DEMO están rodeados por vecinos de las clases mayoritarias. En cualquier vecindario de tamaño 5, la probabilidad de que la mayoría de los vecinos pertenezca a una clase grande es muy alta, así que la votación casi siempre favorece a NUEVO o FLOTILLA. Es la

limitación conocida del algoritmo ante desbalance severo, y es la razón por la que reportar solo el accuracy global habría ocultado el problema por completo.

7. Conclusiones

El algoritmo **aprende del dataset y realiza predicciones correctas**, cumpliendo el objetivo de la actividad. Con 81.51 % de accuracy frente al 43 % de la línea base trivial, el modelo captura patrones reales de la cartera y no está adivinando.

El análisis por clase muestra que el techo del desempeño lo imponen **las variables disponibles y no la implementación**. Todo lo que puede distinguirse a partir de monto, tasa y antigüedad se clasifica muy bien (F1 superior a 0.95 en REFACC y SEMINUEVOS); lo que requiere información ausente del dataset, como la diferencia entre NUEVO y FLOTILLA, se confunde de manera consistente y explicable. Un error sistemático y comprensible es preferible a uno aleatorio, porque señala con precisión qué haría falta para mejorar.

Las dos vías de mejora que se desprenden del análisis son claras. Primero, **incorporar variables sobre el cliente** (tipo de persona, número de unidades contratadas) atacaría directamente el 77 % de los errores. Segundo, **balancear las clases** mediante ponderación del voto por distancia o submuestreo de las clases mayoritarias mejoraría el recall de DEMO, hoy la debilidad más marcada del modelo.

Desde el punto de vista de la implementación, escribir el algoritmo manualmente hizo evidente que en KNN las decisiones de preprocesamiento pesan tanto como el algoritmo en sí: sin la normalización a puntuación z el modelo colapsaría, porque una sola variable dominaría la métrica de distancia.

8. Reproducción

El programa se ejecuta con un solo comando, sin notebook ni IDE, desde cualquier directorio:

```
python3 main.py
```

Imprime en consola el tamaño de los conjuntos, el accuracy y las métricas por clase, y despliega la matriz de confusión y la gráfica de métricas. Requiere Python 3 y matplotlib (solo para graficar). La semilla fija garantiza que los números de este reporte se reproduzcan de forma idéntica.

Archivo	Contenido
main.py	Implementación completa: carga, partición, normalización, KNN, métricas y gráficas
data/interest_ledger.csv	Dataset de 10,576 renglones
docs/reporte.pdf	Este documento